

ReadmitRisk

Predicting when a hospital patient is likely to come back, using survival analysis done the right way

A data science and machine learning case study by Linga Reddy Gudisha

In one sentence

I built a complete machine learning system that estimates how soon a hospital patient is likely to be readmitted, draws an easy to read risk curve for each person, explains what is driving that risk, and checks itself for fairness across different groups of patients. Everything runs locally on safe synthetic data, and a reviewer can start the whole thing with a single command.

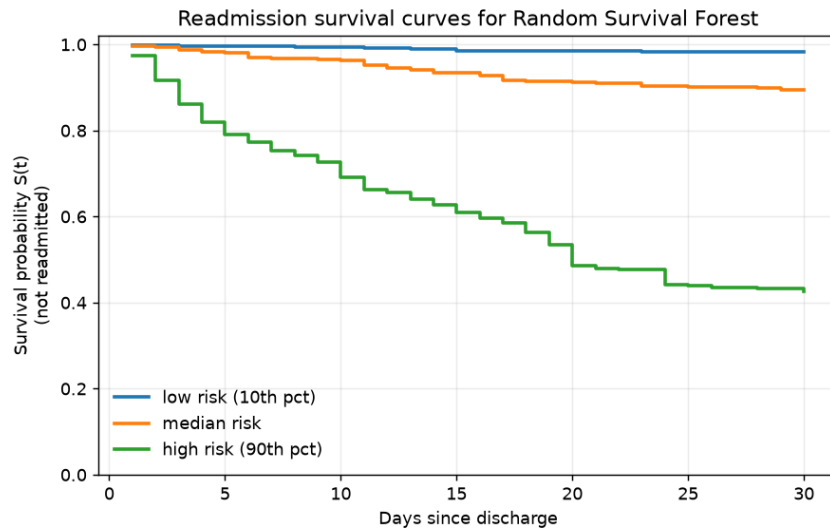
The problem it solves

When a patient leaves the hospital, a big question is whether they will get sick again and return soon. If that happens inside 30 days it is called a readmission, and hospitals are financially penalized for having too many of them. Care teams want to know ahead of time which patients are most at risk, and importantly when that risk is highest, so they can plan follow up visits and avoid an expensive, unsafe return.

The key idea that most projects get wrong

Most beginner projects answer a simple yes or no question: will this patient be readmitted? The trouble is that when our observation window ends, many patients simply have not come back yet. We genuinely do not know their future. In statistics these patients are called censored. A yes or no model is forced to label them a no, which is a guess dressed up as a fact, and it quietly corrupts the results.

The correct approach is survival analysis, the same family of methods used to study how long a treatment keeps working. It understands the difference between a patient who did not return and a patient we simply stopped watching in time. Handling that distinction properly is the core of this project, and I even wrote an automated test that refuses to let anyone slip in the misleading yes or no accuracy score.



Each line is one patient. It shows the chance they stay out of the hospital over the 30 days after discharge. Lower lines mean higher risk.

How it works, step by step

Think of it as a six station assembly line. Raw data goes in one end and an interactive dashboard comes out the other.

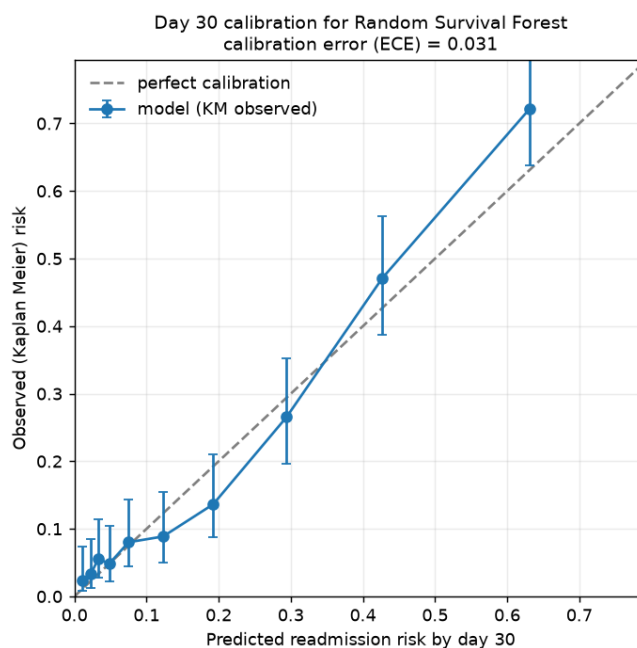
- **1. Make realistic pretend patients.** Real hospital data needs months of legal approval and carries privacy risk, so the project generates its own synthetic patients. No real people are involved.
- **2. Build the timeline.** Using a fast database called DuckDB, I wrote careful medical logic in SQL to find each hospital stay, measure the time until the next visit, mark whether the patient returned within 30 days or whether we ran out of watching time, and gather every useful detail. This is the data engineering heart of the project.
- **3. Train a classic model.** A Cox proportional hazards model, trusted in medicine and easy to explain.
- **4. Train a stronger model and grade both.** A Random Survival Forest, then proper survival report cards for both, with an honest passing bar.
- **5. Check for fairness.** Split the results by sex, age band, race, and ethnicity, and flag any group the model serves noticeably worse.
- **6. Build the dashboard.** A clean web app to pick a patient, see their risk curve, see what is driving it, and read the fairness report.

Results, reported honestly

I scored the models with proper survival measures only. The headline is the concordance score (also called the C index), where 0.5 is a coin flip and 1.0 is perfect.

Model	Concordance	IPCW concordance	Integrated Brier	Calibration error
Random Survival Forest	0.814	0.817	0.083	0.030
Cox Proportional Hazards	0.810	0.812	0.089	0.046

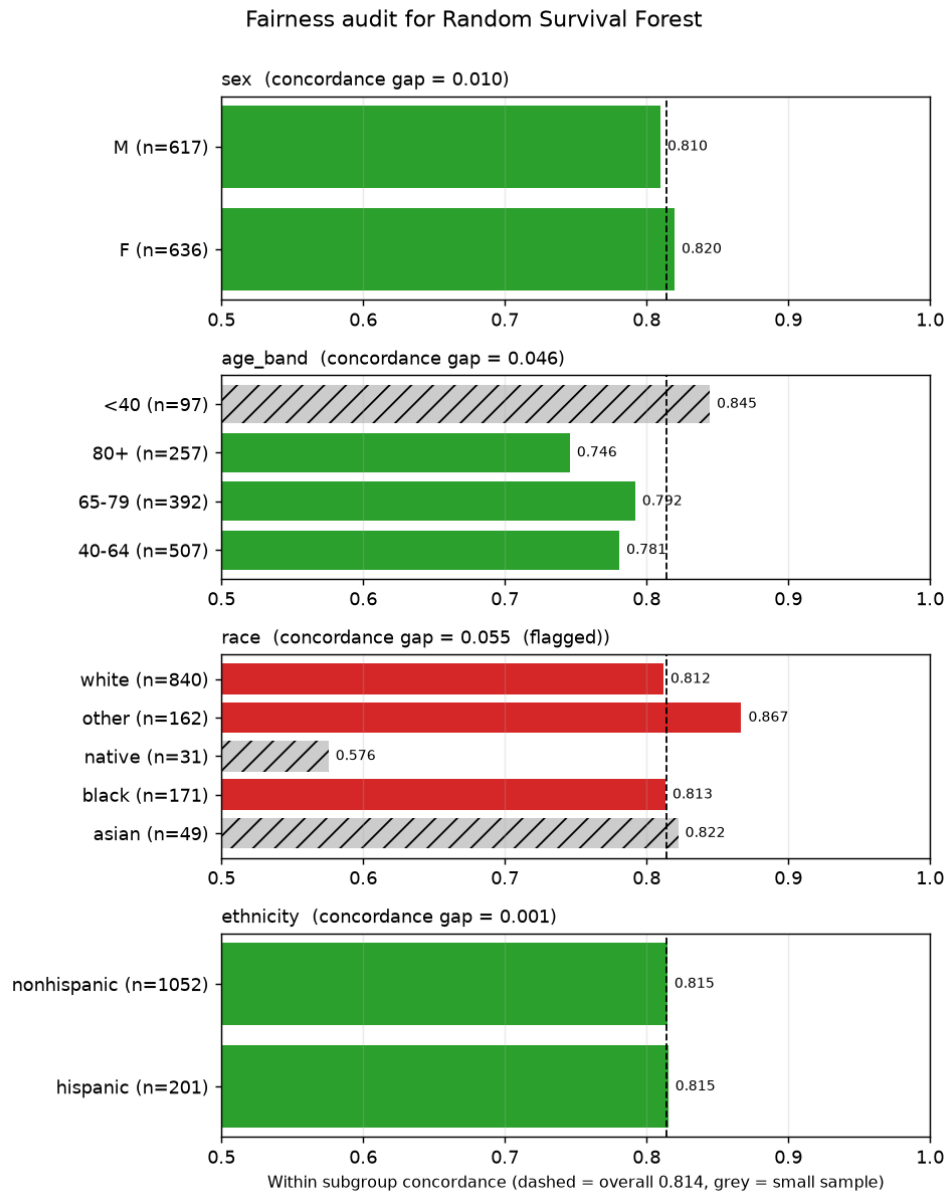
Lower integrated Brier is better, where 0.25 would mean useless. The project requires a concordance of at least 0.70 to pass, and it clears it comfortably. The model is also well calibrated, meaning its stated risks match what actually happens.



Calibration check. Points near the dotted line mean the predicted risk matches the real, observed risk.

Fairness, treated as a first class concern

A model can look strong overall and still work unevenly for different groups of people. I measured the concordance and calibration separately for each group and flagged any gap beyond a set threshold. I was also careful with the statistics: a group with very few events gives a noisy number, so those are reported for transparency but kept out of the flagging. This kind of responsible AI thinking is exactly what teams in healthcare and finance ask about today.



Concordance within each subgroup. Grey bars are small samples shown for transparency but excluded from the fairness flag.

Tools I used

Python	The main programming language
DuckDB and SQL	Fast database where I wrote the medical timeline logic
lifelines and scikit survival	The two survival models and the scoring
SHAP	Explains why the model gave each patient their score
Streamlit and Plotly	The interactive dashboard and charts
pytest, Docker, GitHub Actions	Automated tests, one command setup, and continuous checks

What this project shows about me

- **Real data engineering.** The heavy lifting is careful clinical SQL, not a tidy spreadsheet handed to me.
- **Proper statistics.** I used a harder, rarer method (survival analysis) and handled censoring correctly, which most people miss.
- **A full system, not a notebook.** Data generation, two models, rigorous evaluation, a fairness audit, tests, Docker, and continuous integration.
- **Responsible AI.** A built in fairness audit with honest caveats about small samples.
- **Honesty.** I report the real numbers and state the limitations plainly, which is what builds trust with a hiring team.
- **Product sense.** Anyone can open one link and explore a patient's risk in minutes.

Code: github.com/lithin45/readmitrisk | **Author:** Linga Reddy Gudisha

Note on safety: this is a self contained demo built on fully synthetic data. There are no real patients, no external services, and no keys. It is free to run and safe to share.